Seurat_tutorial

Meryem Stroosma

2025-12-11

Content

Introduction

This RMarkdown contains a brief summary of the R package Seurat. Seurat is used for the analysis of single-cell RNA sequencing data, providing tools for data preprocessing, quality control, normalization, dimensionality reduction, clustering, differential expression, and cell-type annotation. Besides analyzing a dataset Seurat can also be used to create a variaty of plots to visualize the data. The summary in this RMarkdown is based on the Seurat guided clustering tutorial.

Purpose

The purpose of this RMarkdown is to understand the different functionalities of Seurat, and to learn how to use Seurat for analyzing the GSE138852 count matrix.

Data

The data that is used in this RMarkdown can be accessed here. This is a dataset of Peripheral Blood Mononuclear Cells (PBMC) freely available from 10X Genomics. This dataset is also used in the Seurat guided clustering tutorial.

Packages

Prior to analysis several packages need to be loaded. The required packages are the following:

- `dplyr`: this package is used by Seurat for data manipulation.
- `Seurat`: this package needs to be loaded prior to running any Seurat specific functions.
- `patchwork`: the patchwork package can be used to combine several plots into a single figure.
- `here`: this package is used to make sure the file paths are reproducible.

Loading the dataset and creating a Seurat object

In order to perform analysis with Seurat, the data first needs to be loaded. This is done with `Read10X()` which is a Seurat function that can be used to load 10X genomics files such as matrices and .tsv files. After loading the dataset, the function `CreateSeuratObject()` is used to create a Seurat object that contains the raw data. In this object several filters can be applied or altered, which is shown in the following sections.

Pre-processing of the dataset

To make sure the data that will be analyzed is reliable it is important that low quality cells are removed from the dataset. In this case the quality doesn't refer to a Phred score (read quality), but to the amount of unique features per cell, the percentage of mitochondrial genes and the total number of molecules per cell;

- The number of unique features per cell are relevant since too few features compared to the rest of the cells could mean that sequencing of this particular cell was insufficient or that the droplet did not contain a cell. If the number of features is higher it could mean that two or more cells were captured in the same droplet. In most cases a feature is a gene, but this doesn't always have to be the case, sometimes this might be regulatory RNA instead.
- The number of counts says something about the number of different molecules or Unique Molecular Identifiers (UMIs) in one cell. If this number is too low it could mean that this is a poor capture and if this number is too high it could mean that there might be more then one cell in a droplet.
- The percentage of mitochondrial genes are a good indication about the general health of the cell, if this percentage is high it usually means that the cell is dying or stressed. Since it might not be representative for the condition that is analysed it is better to filter this out.

The first step in pre-processing is adding a column to the dataframe with the percentage of mitochondrial genes. To give a general idea of what the dataframe looks like after adding this column the first five rows are shown in the chunk below. In the first column the cell barcodes are shown, the second column contains the original identity of the sample the third, fourth and fifth column contain the number of the number of counts, unique genes and the percentage of mitochondrial genes.

```
##                  orig.ident nCount_RNA nFeature_RNA percent_mt
## AAACATACAACCAC-1     pbmc3k       2419          779  3.0177759
## AAACATTGAGCTAC-1     pbmc3k       4903         1352  3.7935958
## AAACATTGATCAGC-1     pbmc3k       3147         1129  0.8897363
## AAACCGTGCTTCCG-1     pbmc3k       2639          960  1.7430845
## AAACCGTGTATGCG-1     pbmc3k        980          521  1.2244898
```

The dataframe can also be visualised in a plot, in figure 1 a violin plot is shown of the following metrics;

- **nFeature_RNA**: this plot shows the unique number of genes
- **nCount_RNA**: this plot shows the total amount of RNA molecules detected
- **percent_mt**: this plot shows the percentage of mitochondrial genes In the violin plots in figure 1 each dot represents the datapoint for one single cell, on the x-axis the identity is shown, on the y-axis the amount is shown of the metric that is plotted.

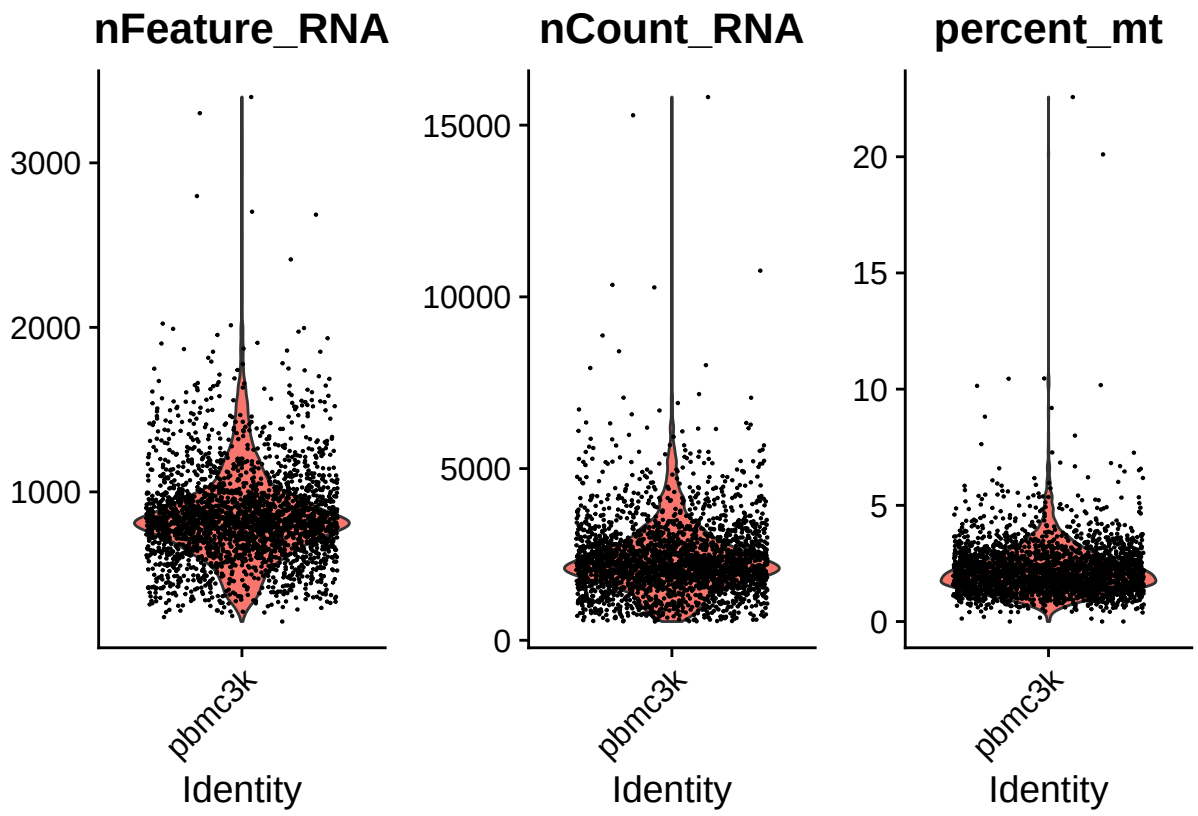

Figure 1: Violin plot of QC-metrics, in the first plot the unique number of genes per cell are visualised, in the second plot the amount of RNA molecules is shown and in the last plot the percentage of mitochondrial genes is shown

With the function `FeatureScatter()` a scatterplot is generated of two QC metrics. The first argument should contain the Seurat object where the QC metrics are stored. Then the x-axis should be defined with the argument `feature1 =`, in figure 2 this was defined as nCount_RNA. The y-axis can be defined in a similar way using `feature2 =`, in the left plot in figure 2 this was defined as *percent_mt* and in the right plot this was defined as *nFeature_RNA*. The title of the plot is the correlation value, this could be changed by the argument plot.cor, which can be set to TRUE or FALSE. `FeatureScatter()` also has several arguments that could be used to modify the plots, for example if the dataset contains cells with different identities these could be grouped in different colours with argument `group.by`.

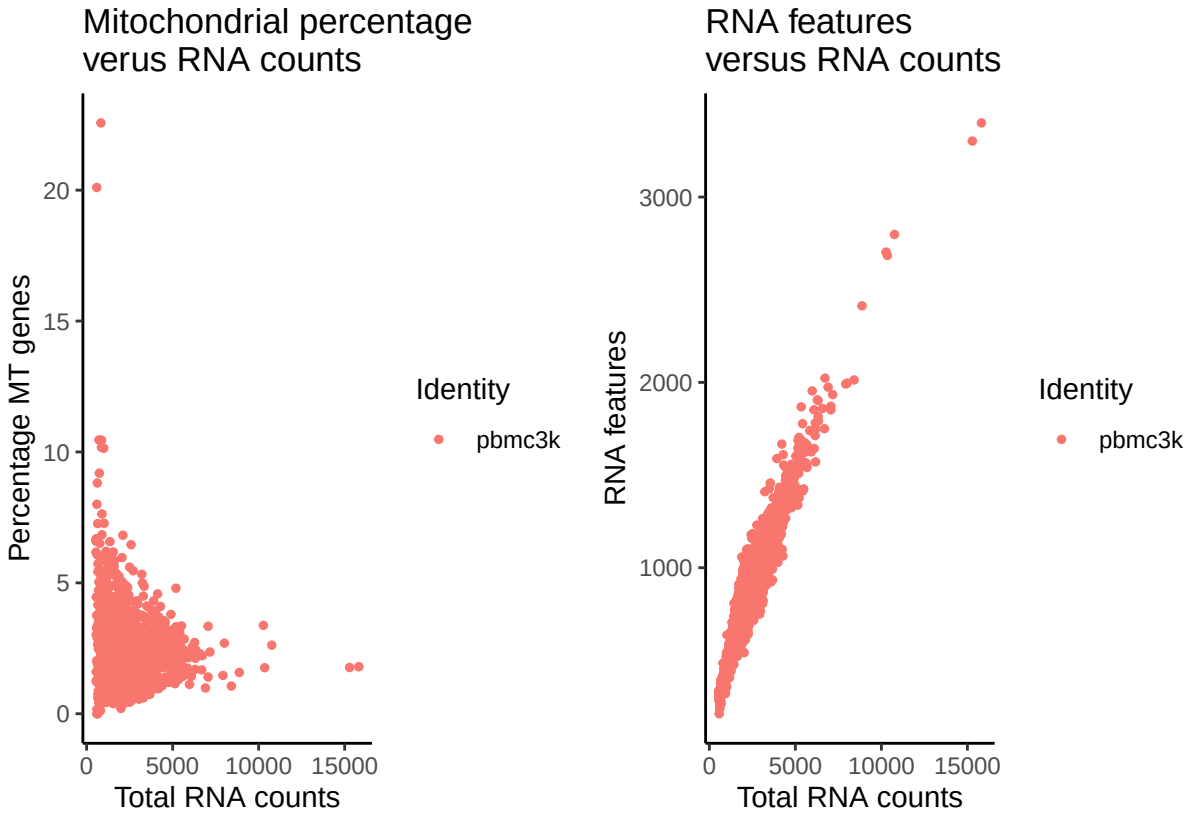

Figure 2: Scatterplot to visualise a relationship between the percentage of mitochondrial RNA versus the total amount of RNA and to visualize the relationship between the total amount of features and the total amount of RNA

The plots in figure 1 and figure 2 contain the unfiltered data of the pbmc dataframe. These plots can help to figure out which cells to use in further analyses and which to filter out. To give an idea of how the plots in figure 1 and figure 2 would look after filtering the data the pbmc dataframe was plotted again after applying the following filters:

- **nFeature_RNA**: Only cells with 200 - 1500 unique features are shown. This means `nFeature_RNA` should be between 200 and 1500 for each cell.
- **percent_mt**: Only cells that express less then 5 percent mitochondrial genes are shown. This means `percent_mt` should be < 5.

The plots of the filtered pbmc dataframe are shown in figure 3 and figure 4.

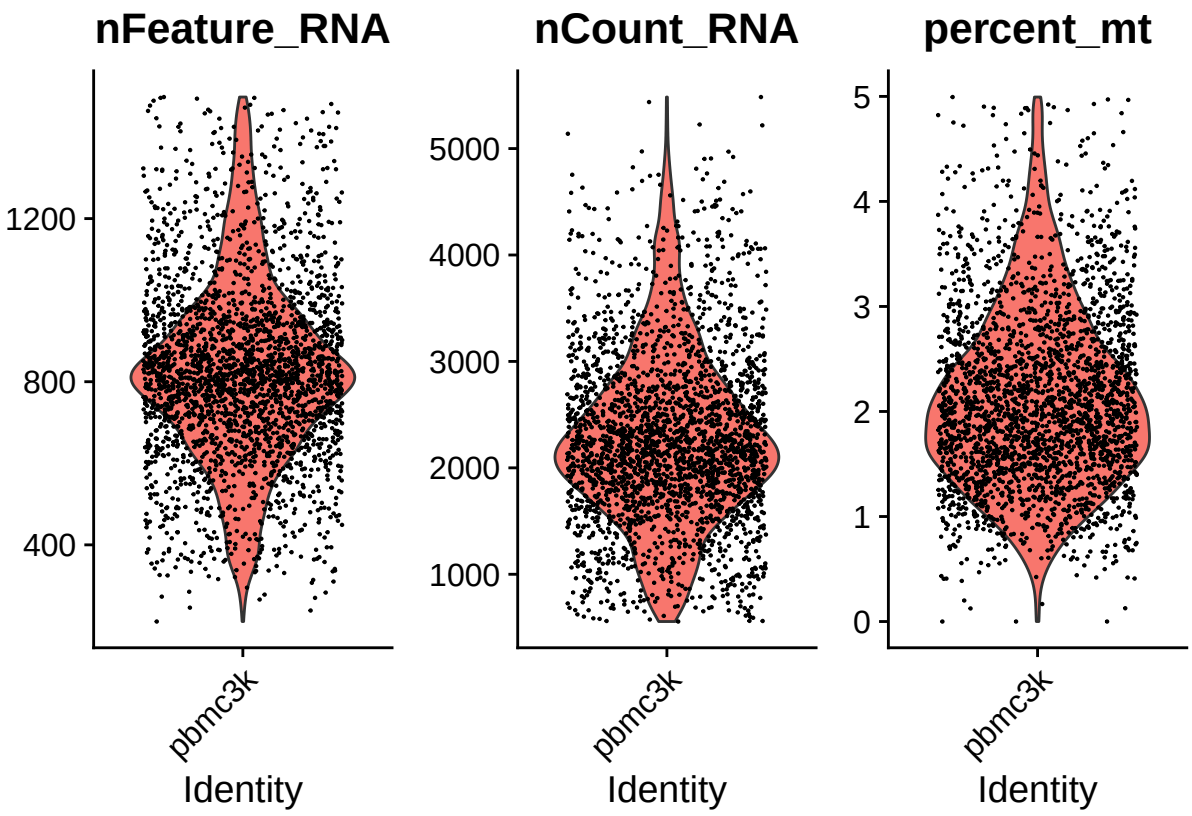

Figure 3: Violin plot of filtered QC-metrics. In the first plot the unique number of genes per cell is visualised, in the second plot the amount of RNA molecules is shown and in the last plot the percentage of mitochondrial genes is shown.

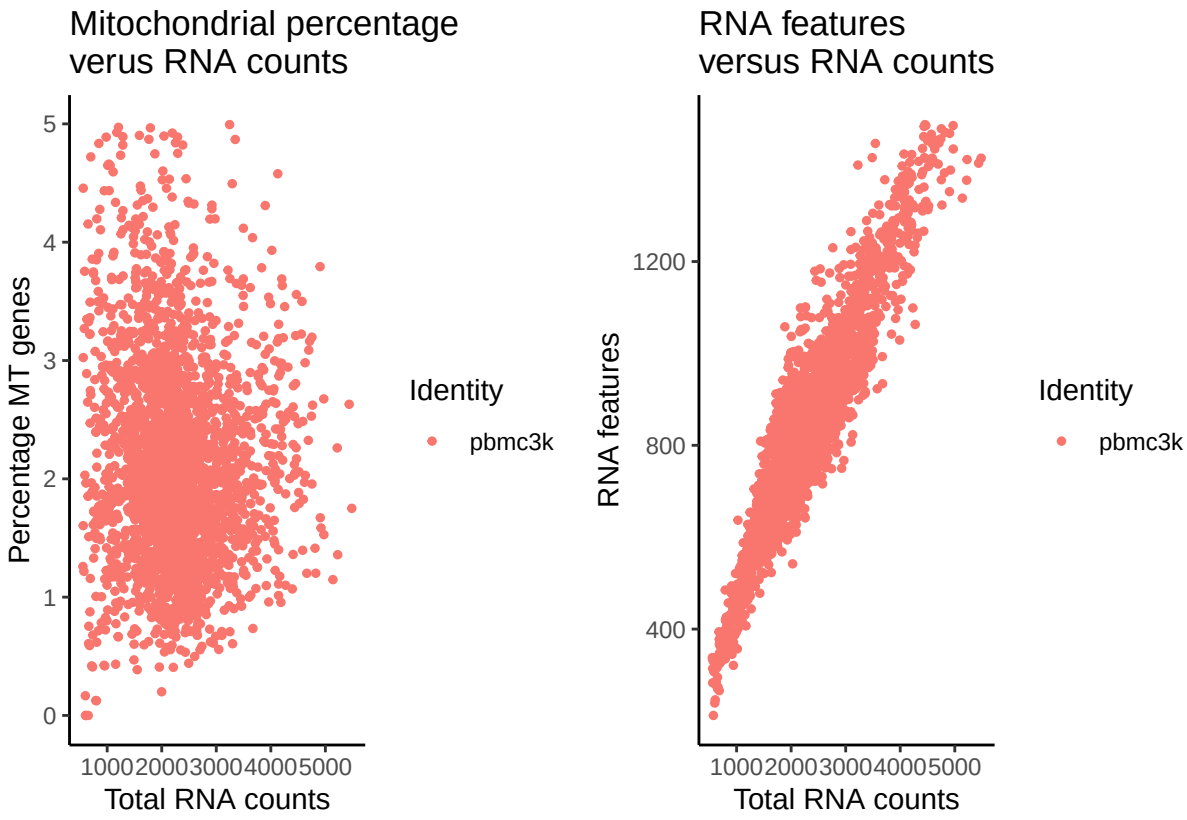

Figure 4: Scatterplot of filtered data to visualise a relationship between the percentage of mitochondrial RNA versus the total amount of RNA and to visualize the relationship between the total amount of features and the total amount of RNA

Normalizing the dataset

In order to correct for differences in sequencing depth between different cells, dropout events and other differences such as total RNA content per cell a normalization step is required. By normalizing the data, biological differences are detected in a more reliable fashion. There are several methods for normalizing the data, quite often a Log normalization is used and in Seurat this is also an option. The function that can be used to do this in Seurat is `NormalizeData()`, this function comes with several arguments. For analyzing differential expression in RNA-seq data `LogNormalize` is the most suitable argument.

- `LogNormalize` : With this argument feature counts per cell are divided by the total counts and multiplied by the `scale.factor`, which is the value to which each cell's total UMI count will be normalized to. De default value for scale.factor is 10000 but can be changed if needed. This method could be used to normalize for sequencing depth and works for all the cells in the sample, this method is typically used since this way of normalizing makes all the cells comparable by scaling the total counts to the same value.

Identification of highly variable features

When analyzing gene expression in general there are thousands of different features that are found. Not all of these features are relevant when analyzing differential gene expression patterns and besides that, using all of these features makes analysis more complicated and time consuming. To perform a reliable and efficient analysis, reducing the amount of features (variables) is a good solution. To determine which variables to analyze and which variables are less relevant the function `FindVariableFeatures()` could be used in Seurat. `FindVariableFeatures` has several different methods to determine the top variable features:

- `vst`: Vst is short for Variance Stabilizing Transformation. This is the default setting of FindVariableFeatures. Vst looks at the expression level and then finds features whose variance is higher than expected in relation to the mean expression of that feature. This is useful since the relative variance is usually higher for genes that have a lower expression pattern, while this is usually lower for genes that have a higher expression pattern. Vst works in several steps, first it takes the *log(variance)* and *log(mean)* and fits a curve using logical polynomial regression (LOESS). This curve makes an estimation of the expected variance given the mean expression of a feature. Then it calculates standardized variance for each feature by *standardized variance (observed variance - expected variance) / expected variance*. Eventually it decides based on the largest values for standardized variance wich features are highle variable.
- `mean.var.plot` : Mean variance plot or MVP is an older method compared to vst. It works by calculating average expression and dispersion. Then it divides features into bins by looking at the average expression and it calculates a Z-score for dispersion *dispersion = variance / mean* per bin. The number of bins can be adjusted. This is based on the idea that genes with the same mean expression should have a similar technical variance, when the expression of a gene is different from the mean expression it should be detected with MVP.

- `dispersion` : Dispersion is the simplest method in Seurat to identify variable features. Dispersion is the *variance* divided by the *mean*, this method just looks at the highest dispersion without adjusting for mean expression. Dispersion is therefore the least accurate method since the mean expression could strongly influence the dispersion value.

The default method in `FindVariableFeatures` is `vst`, which is also the method that will be used in the next steps.

Data scaling

To prepare the dataset for Principal Component Analysis (PCA), first scaling of the data needed. Data scaling is performed by centering each feature to have a mean of 0, then each gene variance is scaled to 1 and optionally some variables could be regressed out. If a feature is high in expression, variance is naturally higher. If a feature is low in expression, variance is naturally lower. By dividing each feature by its standard deviation it is ensured that each gene contributes to PCA without bias. Scaling the data in this way ensures that biological differences are detected in PCA instead of only features that have high expression patterns.

In Seurat the function `ScaleData()` is used to scale the data, in this function several arguments could be used to decide which variables to filter.

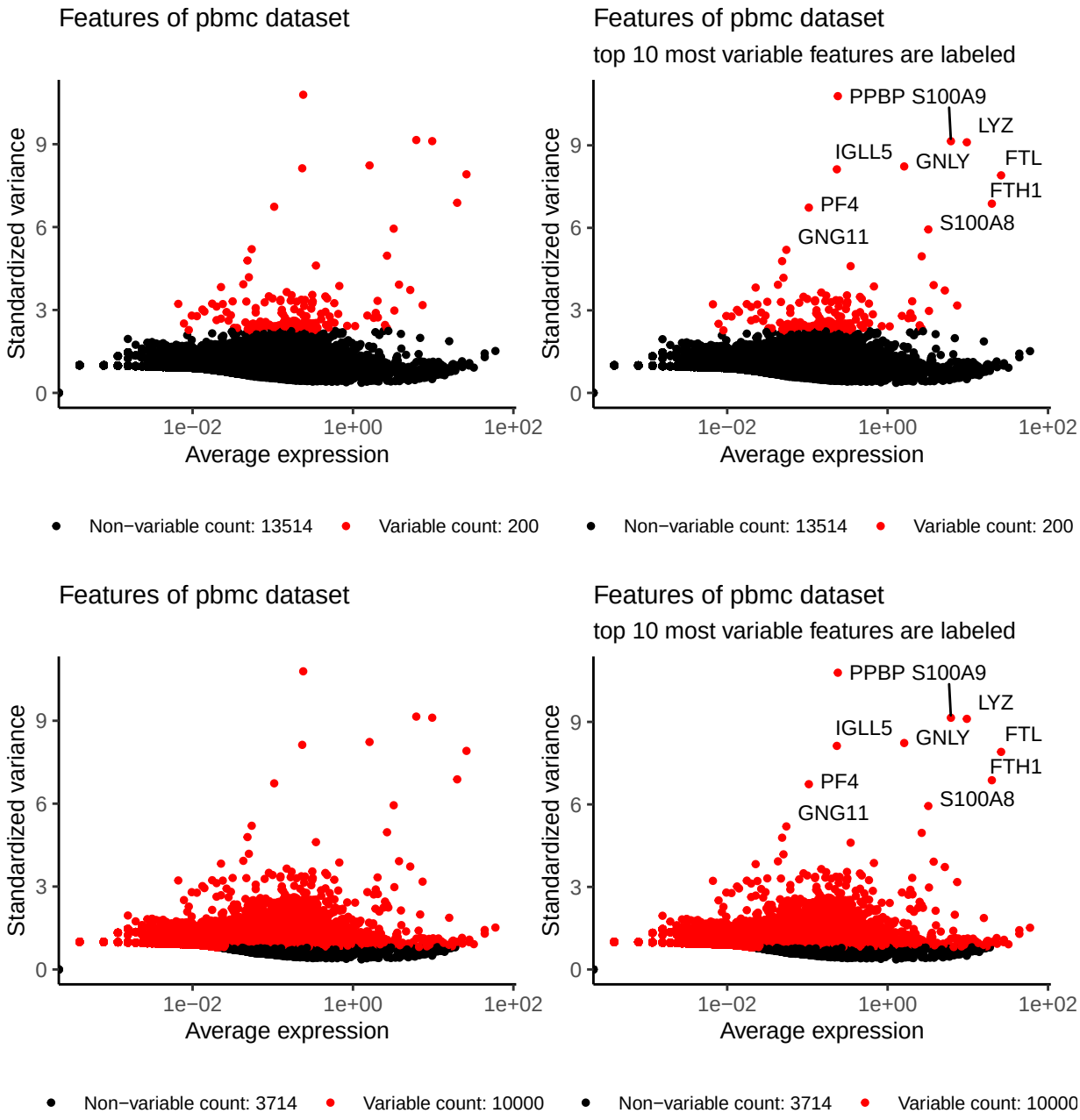

Figure 5: Scatterplot of variable features with and without labels, in the top two plots the nFeatures was set at 200, for the bottom two plots nFeatures was set at 10000 features.

Linear Dimension Reduction

After scaling the data the output of `ScaleData()` could be used to perform Principal Component Analysis (PCA). PCA is a method that could be used for dimension reduction if you have a dataset with a lot of different variables. In the context of gene expression analysis all the different features are considered variables, the amount of variables in such a dataset are enormous. In order to analyse as much data as possible without looking at potentially irrelevant data PCA is performed. PCA does not filter the data directly, but instead it re-formats the same amount of data in a smaller number of components. For example PCA could reduce the amount of variables from thousands of features to 50 Principal Components (PCs). Then it ranks the PCs from largest source of variation (PC1) to smallest source of variation (PCx).

In Seurat PCA is performed with the function `RunPCA()`, within this function "`npcs`" could be used to determine how many PCs should be computed.

An example of the data stored in the object created by RunPCA is shown below. In the first line the PC is given, the second line shows the first five positive features and the third line shows the first five negative features. Here it is also visible that changing the filter on FindVariableFeatures has an effect on the composition of PCs.

```
## PC_ 1
## Positive:  CST7, CCL5, STK17A, NKG7, PRF1
## Negative:  CST3, S100A9, LYZ, FTL, FCN1
## PC_ 2
## Positive:  NKG7, GZMB, FGFBP2, PRF1, CST7
## Negative:  CD79A, TCL1A, HLA-DRA, IGLL5, HLA-DMA
## PC_ 3
## Positive:  CD79A, FCN1, TCL1A, STK17A, RBM39
## Negative:  PPBP, PF4, SDPR, SPARC, GNG11
## PC_ 4
## Positive:  GIMAP5, CORO1B, IFI27, COTL1, GPR183
## Negative:  CD79A, CD74, TCL1A, HLA-DRA, HLA-DMA
## PC_ 5
## Positive:  FCGR3A, ABI3, C1QA, CCT7, NDUFA12
## Negative:  S100A8, MS4A6A, LGALS2, S100A9, CCL3

## PC_ 1
## Positive:  MALAT1, RPS27A, RPL3, RPS3A, RPSA
## Negative:  CST3, TYROBP, LST1, FTL, AIF1
## PC_ 2
## Positive:  NKG7, PRF1, GZMB, CST7, GZMA
## Negative:  CD79A, HLA-DRA, MS4A1, HLA-DQA1, HLA-DQB1
## PC_ 3
## Positive:  CD3D, LDHB, VIM, RPS12, RPL10
## Negative:  CD79A, CD79B, MS4A1, HLA-DQA1, TCL1A
## PC_ 4
## Positive:  PPBP, PF4, SDPR, SPARC, GNG11
## Negative:  RPS19, RPS2, RPL10, RPL19, HLA-DQA1
## PC_ 5
## Positive:  PF4, SDPR, SPARC, PPBP, HIST1H2AC
## Negative:  S100A8, LGALS2, NKG7, S100A9, FGFBP2
```

To visualize the PCA Seurat provides several ways to plot the data:

- **Principal component feature loading plot**: This plot shows which genes are contributing the strongest to a PC. On the x-axis of this plot it shows if a gene has a positive or a negative loading.

This doesn't say anything about how strongly a gene is expressed, but genes with high positive or negative loadings display opposing gene expression programs. It could mean that the genes with a negative loading are expressed by one particular type of cell, while a different type of cell is expressing the genes with a positive loading. In Seurat this plot is created by `VisDimLoadings()`. In figure 6 and figure 7 examples are shown.

- **PCA cell embedding plot**: This plot shows the distribution of cells within PCs. For example PC1 could be on the x-axis and PC2 on the y-axis, the cells are then plotted based on their coordinates for PC1 and PC2. In Seurat `VizDimLoadings()` can be used to create this plot. In figure 8 and figure 9 examples are shown.
- **PCA Heatmap**: A heatmap shows the top genes contributing to a PC across a population of cells. In Seurat `DimHeatMap` can be used to create a heatmap. Several arguments can be used to customize the heatmap, for example to change the amount of dimensions and cells that are plotted.

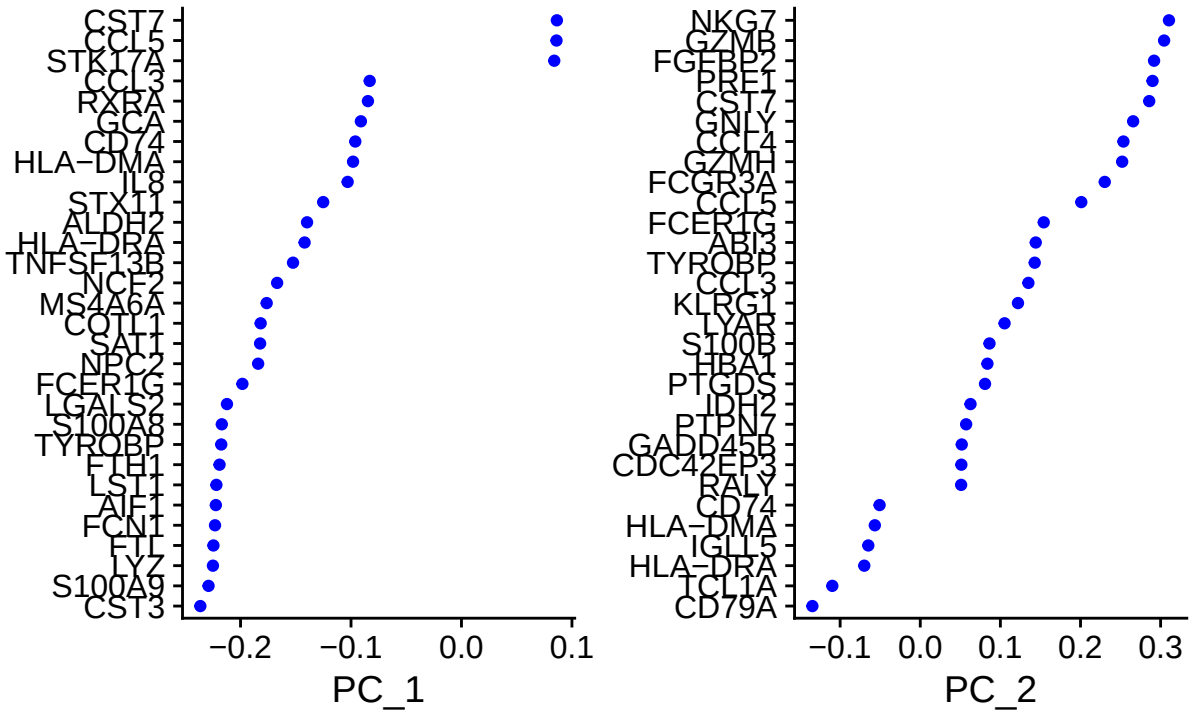

Figure 6: Principal Component feature loading plot, this plot shows which features contribute most to PC1 and PC2. This plot is created with nFeatures = 200.

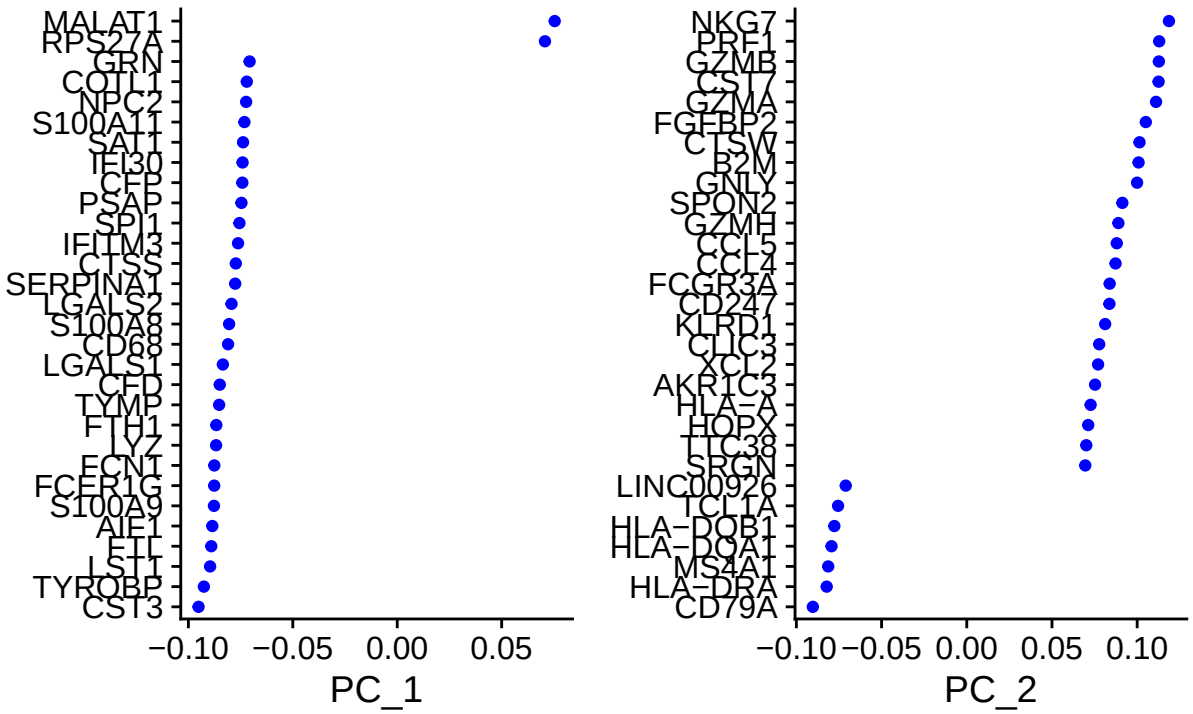

Figure 7: Principal Component feature loading plot, this plot shows which features contribute most to PC1 and PC2. This plot is created with nFeatures = 10000.

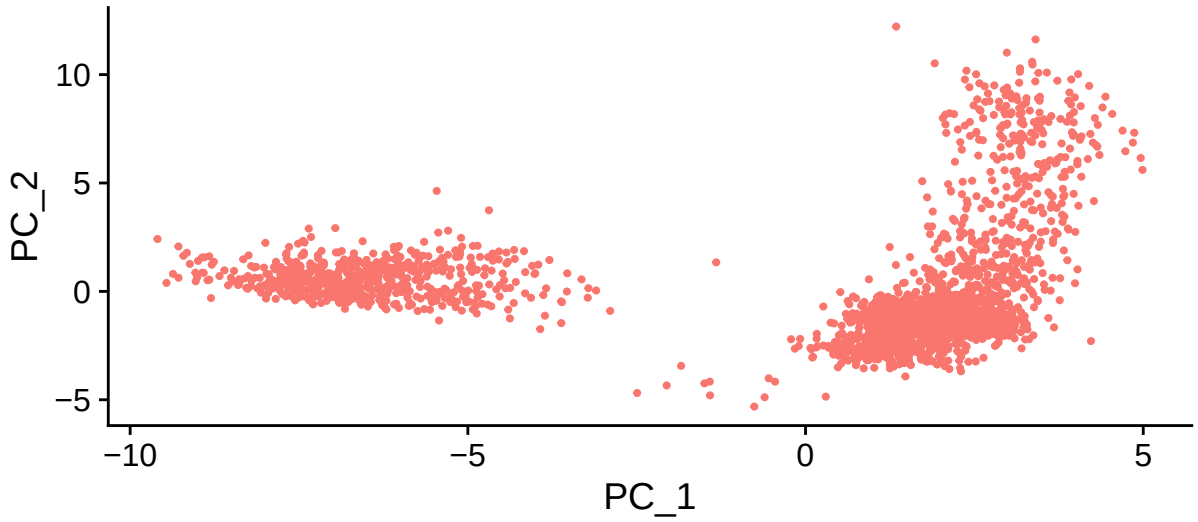

Figure 8: PCA cell embedding plot for the object with nfeatures = 200. Each dot represents a cell positioned by its PC1 and PC2 coordinates.

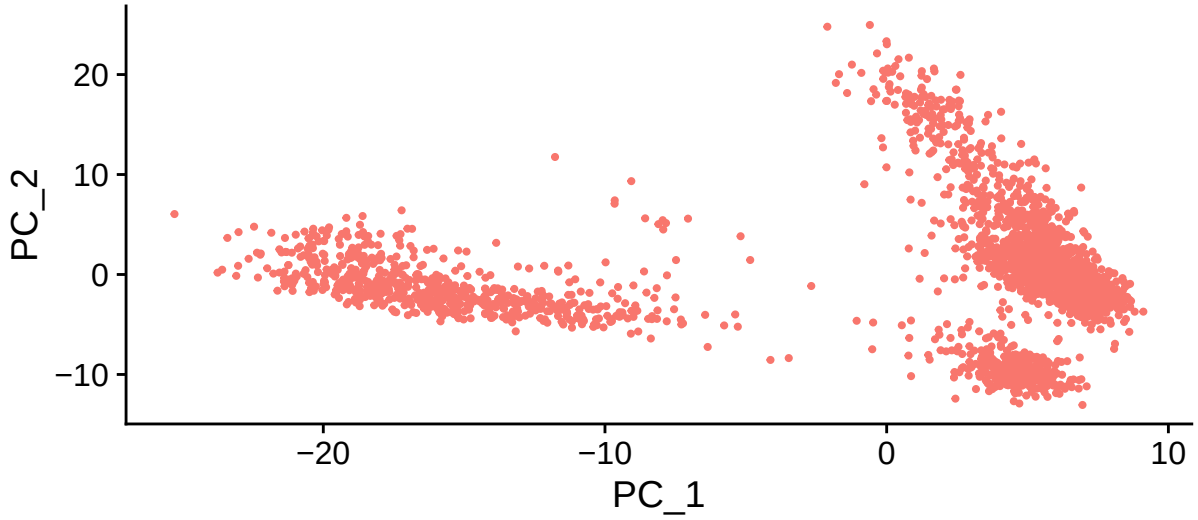

Figure 9: PCA cell embedding plot for the object with nfeatures = 10000. Each dot represents a cell positioned by its PC1 and PC2 coordinates.

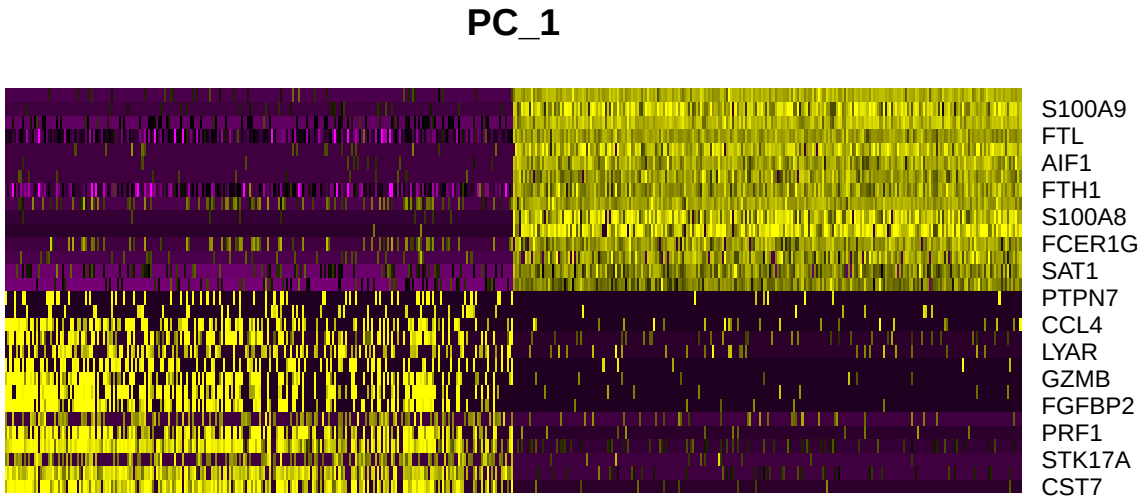

Figure 10: Principal component heatmap. This heatmap is showing the top genes contributing to PC1 across 500 cells. This heatmap is for object with nfeatures = 200, ncells =500.

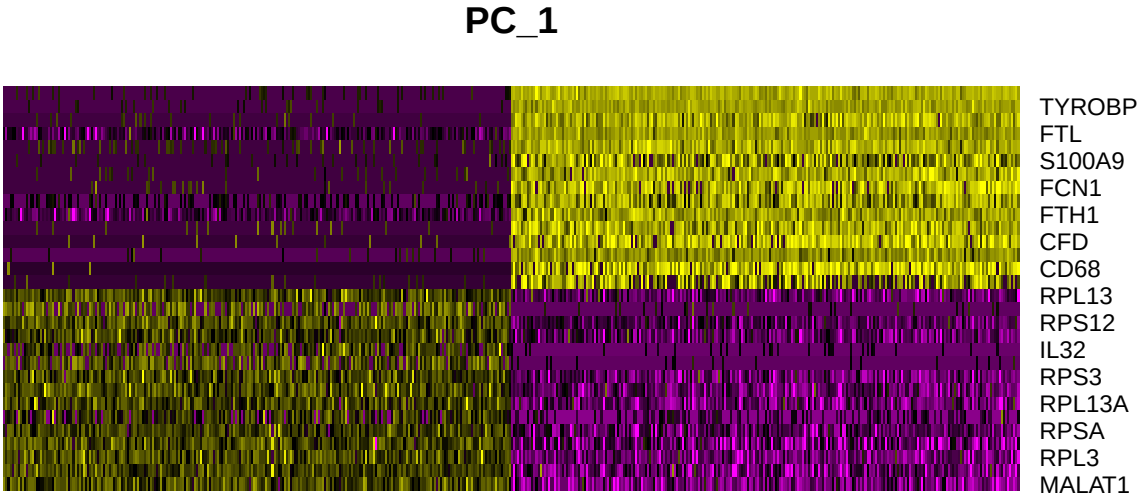

Figure 11: Principal component heatmap. This heatmap is showing the top genes contributing to PC1 across 500 cells. This heatmap is for object with nfeatures = 10000, ncells =500.

Determine dimensionality of the dataset

To see which PCs are really contributing to biological variance and which PCs are potential technical noise, the PCs are plotted in an elbowplot. In this plot can be created in Seurat with `ElbowPlot()`. In this plot the standard deviation of each PC is plotted on the y-axis and the different PCs are ranked from largest to smallest variance on the x-axis. The plot indirectly shows variance since *variance = stdev^2*.

Sources

- 1 Seurat Tutorial
- 2 DAUR2 - RNA sequencing part 2
- 3 Data normalization for addressing the challenges in the analysis of single-cell transcriptomic datasets
- 4 Seurat FindVariableFeatures
- 5 Comprehensive Integration of Single-Cell Data, Tim Stuart et al.
- 6 Seurat ScaleData

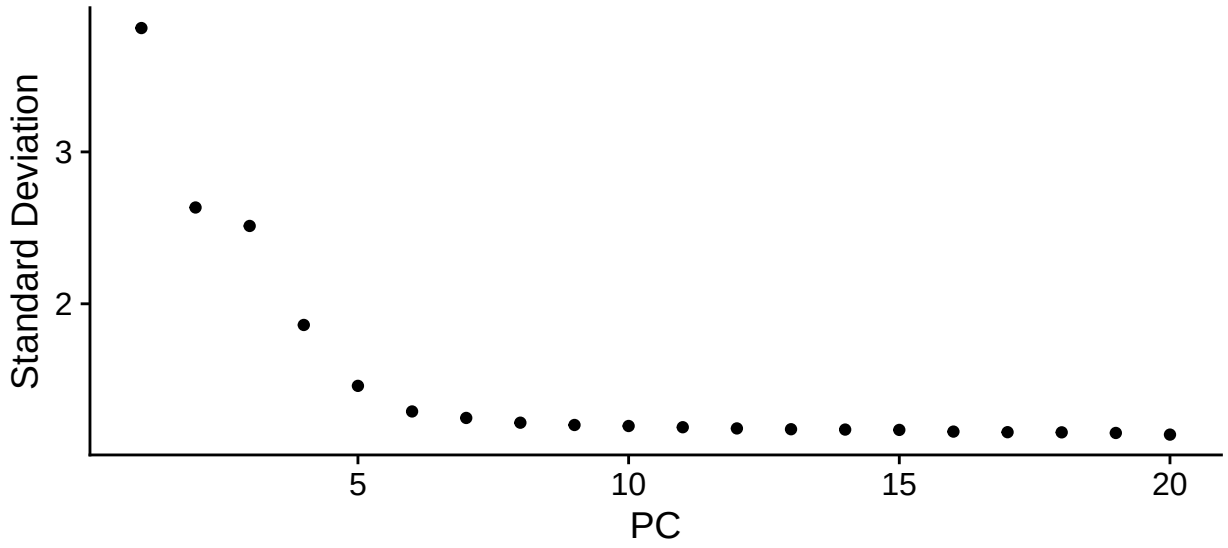

Figure 12: Elbow plot of principal components. This plot shows the standard deviation of each principal component and is used to determine how many PCs capture most of the variation in the data. This elbow plot is based on the object with nfeatures = 100.